

# Djezzy National SOC Platform Deployment & Integration Guide

Complete procedures for deploying and integrating 15+ open-source security tools in a containerized microservices architecture for telecommunications security operations.

# Executive Overview & Architecture

The Djazzy National Security Operations Center (SOC) Platform represents a comprehensive, enterprise-grade solution designed specifically for telecommunications security operations. Built entirely on open-source technologies and deployed as a 100% on-premises solution, this platform integrates **39 microservices** across **15 specialized security tools** to provide complete visibility, detection, and response capabilities for one of Algeria's largest mobile network operators.

The architecture follows a modern **containerized microservices pattern**, utilizing Docker for service isolation and Kubernetes for orchestration at scale. Each security tool operates as an independent container with well-defined API interfaces, communicating through a centralized Apache Kafka event streaming backbone. This design ensures high availability, horizontal scalability, and the flexibility to upgrade or replace individual components without disrupting the entire system.



## Platform Scale Targets

|                           |                                        |
|---------------------------|----------------------------------------|
| Event Ingestion Rate      | 50,000+ events per second (EPS)        |
| Network Monitoring        | 100Gbps throughput capability          |
| Annual Event Retention    | 50 billion+ events per year            |
| Subscriber Capacity       | 15 million+ active subscribers         |
| Security Tools Integrated | 15+ enterprise-grade open-source tools |
| Microservices Count       | 39 containerized services              |

### ✓ Architecture Principle

The platform is designed for **zero cloud dependencies**, making it fully air-gap capable for environments requiring complete network isolation. All components can operate independently of external internet access once initial deployment is complete.

## Core Technology Stack

### Frontend Layer

React 19 based dashboard with TypeScript, real-time SSE updates, and responsive UI using shadcn/ui library.

Next.js 16

### Backend Runtime

High-performance server-side rendering with Prisma ORM for database operations and Redis caching layer.

Node.js 20

### Database Cluster

Primary data store with PostGIS extension for geospatial queries, PgBouncer connection pooling.

PostgreSQL 16

### Event Streaming

3-broker Apache Kafka cluster with Zookeeper for distributed event processing and message queuing.

Kafka 3.x

CHAPTER 2

# Hardware Requirements & Infrastructure

The Djezzy SOC Platform requires a robust hardware infrastructure to meet the demanding performance requirements of a national-scale telecommunications security operation. This section details the minimum and recommended specifications for both the primary production site and disaster recovery location.

## Primary Site Infrastructure (14 Servers)

| Server Role                       | Specification         | vCPUs   | RAM    | Storage              |
|-----------------------------------|-----------------------|---------|--------|----------------------|
| SOC Platform Nodes (x3)           | Dell PowerEdge R750xa | 64 vCPU | 256 GB | 2× 960GB NVMe        |
| Elasticsearch Cluster (x3)        | Dell PowerEdge R740   | 32 vCPU | 128 GB | 4× 1.92TB NVMe       |
| Kafka/Zookeeper (x3)              | Dell PowerEdge R640   | 24 vCPU | 64 GB  | 2× 960GB NVMe        |
| PostgreSQL Primary + Replica (x2) | Dell PowerEdge R740   | 32 vCPU | 128 GB | 2× 1.92TB + 10TB HDD |
| Security Tools Pool (x2)          | Dell PowerEdge R750   | 48 vCPU | 192 GB | 4× 1.92TB NVMe       |
| NSM/Suricata (x1)                 | Dell PowerEdge R750xa | 64 vCPU | 128 GB | 4× 3.84TB NVMe       |

## Disaster Recovery Site (8 Servers)

The DR site maintains a scaled-down but fully functional copy of the primary infrastructure, designed to take over operations within a defined RTO (Recovery Time Objective) of 4 hours and RPO (Recovery Point Objective) of 1 hour for critical data. The DR site operates in active-passive mode with continuous replication from the primary site via PostgreSQL logical replication and Kafka mirror maker.

### Network Requirements

Minimum **10Gbps dedicated link** required between primary and DR sites for replication traffic. For optimal performance during failover scenarios, a **40Gbps or 100Gbps dark fiber connection** is strongly recommended.

## CHAPTER 3

# Software Prerequisites & Environment Setup

Before initiating the deployment process, ensure that all prerequisite software is installed and properly configured on the target servers. This section outlines the exact versions and configurations that have been validated for production use.

## Operating System Requirements



### Base OS Configuration

- **Operating System:** Rocky Linux 9.3 / RHEL 9.3 (Long Term Support)
- **Kernel Version:** 5.14.0-362 or later with realtime patches for NSM nodes
- **Filesystem:** XFS with reflink enabled for efficient snapshots
- **Time Synchronization:** Chrony configured with Africa/Algiers timezone (UTC+1)
- **SELinux:** Enforcing mode with custom SOC policies

## Container Runtime & Orchestration

| Component      | Version  | Purpose                             |
|----------------|----------|-------------------------------------|
| Docker CE      | 24.0.6+  | Container runtime for all services  |
| Docker Compose | v2.24.0+ | Multi-container orchestration       |
| Kubernetes     | 1.28+    | Production orchestration (optional) |
| Helm           | 3.13+    | Kubernetes package manager          |

```
# Install Docker CE on Rocky Linux/RHEL 9 # Remove any existing Docker installations
dnf remove docker docker-client docker-client-latest docker-common \
docker-latest docker-latest-logrotate docker-logrotate docker-engine # Add Docker repository
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo # Install Docker packages
dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin \
docker-buildx-plugin # Enable and start Docker
systemctl enable --now docker # Verify installation
docker --version
docker compose version
```

CHAPTER 4

# Step-by-Step Deployment Procedure

This chapter provides the complete step-by-step procedure for deploying the Djazzy SOC Platform from scratch. Follow these instructions in sequence to ensure a successful production deployment. Each step includes verification commands to confirm successful completion before proceeding to the next phase.

## 1 Generate Secure Credentials

Begin by generating all necessary API keys, passwords, and certificates required by the platform components. The included script creates cryptographically secure credentials and stores them in environment files.

```
# Navigate to project directory
cd /opt/djazzy-soc-platform # Generate secure credentials
node scripts/generate-secure-credentials.js # Review generated credentials file
cat .env.production.filled | head -30
```

## 2 Deploy Infrastructure Layer

Start the foundational services that all other components depend upon: PostgreSQL database, Redis cache, and PgBouncer connection pooler. These services must be fully operational before deploying security tools.

```
# Start infrastructure services in order docker compose -f docker-compose.prod.yml
up -d \ postgresql \ pgbouncer \ redis-master \ redis-replica-1 \ redis-replica-2 \
redis-sentinel-1 \ redis-sentinel-2 \ redis-sentinel-3 # Wait for PostgreSQL to be
ready (health check) docker compose -f docker-compose.prod.yml logs -f postgresql |
grep -q "database system is ready" # Initialize database schema docker compose -f
docker-compose.prod.yml exec postgresql psql -U soc_user \ -d soc_production -f
/docker-entrypoint-initdb.d/01-init-schema.sql
```

### 3 Deploy Event Streaming Layer

Initialize the Apache Kafka cluster with Zookeeper ensemble. Kafka serves as the central nervous system for event distribution between all security tools. A 3-broker configuration provides fault tolerance and horizontal scalability.

```
# Start Zookeeper ensemble first (required by Kafka) docker compose -f docker-
compose.prod.yml up -d \ zookeeper-0 zookeeper-1 zookeeper-2 # Verify Zookeeper is
running for i in 0 1 2; do echo "Checking zookeeper-i..." dockercompose -f docker -
compose.prod.yml exec zookeeper-i \ zkServer.sh status | grep "Mode:" done # Start
Kafka brokers docker compose -f docker-compose.prod.yml up -d kafka-0 kafka-1
kafka-2 # Create required topics docker compose -f docker-compose.prod.yml exec
kafka-0 \ kafka-topics.sh --create --if-not-exists \ --topic soc.alerts --
partitions 12 --replication-factor 3 \ --bootstrap-server localhost:9092 docker
compose -f docker-compose.prod.yml exec kafka-0 \ kafka-topics.sh --create --if-
not-exists \ --topic soc.events --partitions 24 --replication-factor 3 \ --
bootstrap-server localhost:9092
```

### 4 Deploy SIEM Stack (Wazuh + Elasticsearch)

Deploy the Security Information and Event Management stack consisting of Wazuh Manager for correlation/alerting and Elasticsearch cluster for log storage and search. This forms the backbone of the platform's detection capabilities.

```
# Deploy Elasticsearch cluster (3 nodes for HA) docker compose -f docker-
compose.prod.yml up -d \ elasticsearch-node-1 elasticsearch-node-2 elasticsearch-
node-3 # Wait for Elasticsearch cluster to form (green status) sleep 30 curl -k
https://localhost:9200/_cluster/health?pretty # Deploy Wazuh components docker
compose -f docker-compose.prod.yml up -d \ wazuh-indexer wazuh-manager wazuh-
dashboard # Verify Wazuh API is accessible curl -k https://localhost:55000 -u
admin:${WAZUH_PASSWORD}
```

### 5 Deploy EDR Stack (GRR + Osquery Fleet)

Deploy the Endpoint Detection and Response infrastructure including GRR Rapid Response for remote live forensics and Osquery Fleet for distributed endpoint querying. These tools enable deep visibility into endpoint activity across the organization.

```
# Deploy GRR server docker compose -f docker-compose.prod.yml up -d grr-server #
Deploy Osquery Fleet management console docker compose -f docker-compose.prod.yml
up -d osquery-fleet # Initialize GRR administrative account docker compose -f
docker-compose.prod.yml exec grr-server \ grr_config_updater initialize --
external_hostname=grr.djezzy.local # Generate Osquery enrollment secret docker
compose -f docker-compose.prod.yml exec osquery-fleet \ fleetctl setup --env-file
/etc/fleet.env
```

## 6 Deploy NSM Stack (Suricata + Zeek + Arkime)

Deploy the Network Security Monitoring stack for real-time intrusion detection, protocol analysis, and full packet capture. Suricata provides IDS/IPS capabilities, Zeek performs deep protocol analysis, and Arkime enables PCAP visualization and search.

```
# Deploy Suricata IDS/IPS (requires host networking) docker compose -f docker-
compose.prod.yml up -d suricata # Deploy Zeek controller for protocol analysis
docker compose -f docker-compose.prod.yml up -d zeek-controller zeek-worker-1 zeek-
worker-2 # Deploy Arkime for PCAP visualization docker compose -f docker-
compose.prod.yml up -d arkime # Configure Suricata to monitor production interfaces
docker compose -f docker-compose.prod.yml exec suricata \ suricata-update update-
rulesources docker compose -f docker-compose.prod.yml restart suricata
```

## 7 Deploy SOAR Stack (TheHive + Cortex)

Deploy the Security Orchestration, Automation and Response platform. TheHive provides case management and incident tracking while Cortex offers automated analysis through integrated analyzers for threat intelligence enrichment.

```
# Deploy TheHive case management docker compose -f docker-compose.prod.yml up -d
thehive cassandra elasticsearch-thehive # Deploy Cortex analysis engine docker
compose -f docker-compose.prod.yml up -d cortex # Configure Cortex analyzers
(VirusTotal, MISP, etc.) docker compose -f docker-compose.prod.yml exec cortex \
/opt/cortex/tools/add-analyzer.py VirusTotal_3 --auto-start true docker compose -f
docker-compose.prod.yml exec cortex \ /opt/cortex/tools/add-analyzer.py MISP_3_1 --
url http://misp:80 --key ${MISP_API_KEY}
```

## 8 Deploy Threat Intelligence (MISP + OpenCTI)

Deploy the threat intelligence platforms for IOC management and STIX-based threat analysis. MISP serves as the primary IoC sharing platform while OpenCTI provides advanced MITRE ATT&CK mapping and threat actor profiling.

```
# Deploy MISP threat intelligence platform docker compose -f docker-
compose.prod.yml up -d misp mysql-misp redis-misp # Initialize MISP administration
docker compose -f docker-compose.prod.yml exec misp \ ./server/initMISP.sh # Deploy
OpenCTI for advanced TI analytics docker compose -f docker-compose.prod.yml up -d
opencti redis-opencti opencti-connector-worker # Configure OpenCTI data connectors
docker compose -f docker-compose.prod.yml exec opencti-connector-worker \ python
connectors/misp-feed.py --start
```

## 9 Deploy Vulnerability Management (OpenVAS + DefectDojo)

Deploy the vulnerability scanning and management infrastructure. OpenVAS/GVM provides automated vulnerability scanning while DefectDojo offers vulnerability lifecycle management, risk scoring, and compliance reporting.

```
# Deploy OpenVAS vulnerability scanner docker compose -f docker-compose.prod.yml up
-d openvas ospd-openvas gvm gsd # Initialize OpenVAS (this takes ~15 minutes on
first run) docker compose -f docker-compose.prod.yml exec openvas gvm-setup #
Create initial scanning target curl -sSk -u "OPENVAS_USERNAME:
{OPENVAS_PASSWORD}" \ -X POST https://openvas:9392/gmp -d '\
Djezzy Network Infrastructure\ 10.0.0.0/8\ '
```

## 10 Deploy Main SOC Platform Application

Finally, deploy the main Next.js application that serves as the unified dashboard and API gateway connecting all security tools. This application should be started last as it depends on all other services being operational.

```
# Build the SOC platform image docker compose -f docker-compose.prod.yml build soc-
platform # Deploy the main application docker compose -f docker-compose.prod.yml up
-d soc-platform # Wait for health check to pass sleep 60 curl -s
http://localhost:3000/api/health | jq . # Verify all integrations are connected
curl -s http://localhost:3000/api/system | jq '.integrations'
```

## CHAPTER 5

# Integration Procedures & API Configuration

This chapter details the specific integration procedures for each security tool, including API endpoint configuration, authentication setup, and data flow validation. Each integration follows a standardized pattern ensuring consistency and maintainability across the platform.

## SIEM Integration (Wazuh + Elasticsearch)





## Wazuh API Configuration

The Wazuh integration provides real-time alert ingestion, rule management, and compliance monitoring. Configure the following endpoints in your environment variables:

| Endpoint                    | Method   | Purpose                |
|-----------------------------|----------|------------------------|
| /security/user/authenticate | POST     | User authentication    |
| /security/config            | GET      | Manager configuration  |
| /security/events            | GET      | Security events query  |
| /groups                     | GET/POST | Agent group management |
| /manager/info               | GET      | System information     |

```
// src/lib/integrations/siem/wazuh-elasticsearch-client.tsexport interface WazuhConfig {
  apiUrl: 'http://wazuh-manager:55000'; username: process.env.WAZUH_USERNAME; password:
  process.env.WAZUH_PASSWORD; insecure: true; // Internal network only } export interface
  ElasticsearchConfig { nodes: process.env.WAZUH_ES_NODES.split(','); username:
  process.env.WAZUH_ES_USERNAME; password: process.env.WAZUH_ES_PASSWORD; tls: {
  rejectUnauthorized: false }; }
```

## EDR Integration (GRR + Osquery)

### GRR Rapid Response

:8000

Remote live forensics capability supporting memory acquisition, filesystem analysis, and registry examination across 15M+ endpoints.

POST /clients

GET /hunts

POST /flows

### Osquery Fleet

:8080

Distributed SQL-based endpoint interrogation providing real-time process monitoring, file integrity, and configuration auditing.

GET /api/v1/osquery/distributed

POST /api/v1/osquery/differential

## SOAR Integration (TheHive + Cortex)

```
// SOAR Integration Configurationconst soarConfig = { thehive: { apiUrl:
  'http://thehive:9000', apiKey: process.env.THEHIVE_API_KEY, orgName: 'Djezzy-SOC',
  severityMapping: { critical: 4, high: 3, medium: 2, low: 1 } }, cortex: { apiUrl:
  'http://cortex:9001', apiKey: process.env.CORTEX_API_KEY, enabledAnalyzers: [
  'VirusTotal_3', 'MISP_3_1', 'OpenCTI_2', 'AbuseIPDB_2', 'DNSResolver_2_1' ] } };
```

# Threat Intelligence Integration

## 🔒 Threat Intelligence Feed Configuration

Both MISP and OpenCTI are configured with automatic feed synchronization. The platform subscribes to commercial feeds (Recorded Future, Intel 471) and community sources (AlienVault OTX, Abuse.ch). Feeds synchronize every 15 minutes with automatic IOC extraction and scoring.

| Tool    | Feed Sources          | Sync Interval | IOC Types                            |
|---------|-----------------------|---------------|--------------------------------------|
| MISP    | 200+ configured feeds | 15 minutes    | IP, Domain, Hash, URL, Email         |
| OpenCTI | STIX/TAXII servers    | 30 minutes    | STIX 2.1 indicators, TTPs, Campaigns |

## CHAPTER 6

# Post-Deployment Verification

After completing the deployment procedure, execute the following verification steps to ensure all components are properly integrated and functioning correctly. This systematic approach identifies potential issues early before they impact production operations.

## Health Check Verification

```
# Execute comprehensive health check
curl -s http://localhost:3000/api/health | jq . #
Expected response structure { "status": "healthy", "version": "11.1.0", "uptime": 3600.5,
"components": { "database": { "status": "connected", "latency": 2.3 }, "redis": {
"status": "connected", "latency": 0.5 }, "kafka": { "status": "connected", "brokers": 3 },
"integrations": { "siem": "connected", "edr": "connected", "nsm": "connected", "soar":
"connected", "threatIntel": "connected", "vulnerability": "connected" } } }
```

## Integration Connectivity Test

```
# Test each integration endpoint individually# SIEM Integration Test
curl -s http://localhost:3000/api/metrics/siem | jq '.eventsLastHour' # EDR Integration Test
curl -s http://localhost:3000/api/telecom/probes | jq '.activeEndpoints' # SOAR Integration
Test curl -s http://localhost:3000/api/cases | jq '.openCases' # Threat Intel Sync Test
curl -s http://localhost:3000/api/threats | jq '.iocCount' # Vulnerability Scan Test
curl -s http://localhost:3000/api/compliance | jq '.vulnerabilities.critical'
```

## Integration Connectivity Test

```
# Test each integration endpoint individually# SIEM Integration Test curl -s
http://localhost:3000/api/metrics/siem | jq '.eventsLastHour' # EDR Integration Test curl
-s http://localhost:3000/api/telecom/probes | jq '.activeEndpoints' # SOAR Integration
Test curl -s http://localhost:3000/api/cases | jq '.openCases' # Threat Intel Sync Test
curl -s http://localhost:3000/api/threats | jq '.iocCount' # Vulnerability Scan Test curl
-s http://localhost:3000/api/compliance | jq '.vulnerabilities.critical'
```

## Event Flow Validation

### ✓ Event Pipeline Verification

Verify that events flow correctly through the entire pipeline from ingestion to alerting. Use the Kafka consumer to monitor topic activity and confirm events are being processed by all downstream consumers.

```
# Monitor Kafka event consumption docker compose -f docker-compose.prod.yml exec kafka-0 \
kafka-console-consumer --bootstrap-server localhost:9092 \ --topic soc.alerts --from-
beginning --timeout-ms 5000 # Check consumer group lag docker compose -f docker-
compose.prod.yml exec kafka-0 \ kafka-consumer-groups --bootstrap-server localhost:9092 \
--describe --group soc-platform-consumers
```

## CHAPTER 7

# Troubleshooting & Common Issues

This chapter addresses common deployment issues and their resolutions. Each problem includes symptoms, root cause analysis, and step-by-step remediation procedures based on real-world deployment experience.

### Issue: Container Startup Failures

**Symptoms:** Containers repeatedly restarting with exit code 137 or "OOMKilled" status.

**Root Cause:** Insufficient memory allocation or resource limits not configured for production workload.

**Resolution:**

- Check container logs: `docker compose logs [service]`
- Verify memory limits in `docker-compose.prod.yml` match server capacity
- Increase memory reservation for Elasticsearch (minimum 8GB per node)
- Enable swap on host system for burst scenarios



### Issue: Kafka Connection Timeouts

**Symptoms:** SOC Platform unable to connect to Kafka brokers, timeout errors in application logs.

**Root Cause:** Network segmentation between services or Zookeeper ensemble not fully initialized.

**Resolution:**

- Verify all Zookeeper nodes are in "leader/follower" mode
- Check Kafka broker advertised listeners configuration matches internal DNS
- Validate network connectivity: `nc -zv kafka-0 9092`
- Review firewall rules allow inter-container communication on port 9092



### Issue: Wazuh Agent Connection Refused

**Symptoms:** Agents showing as "never connected" in Wazuh dashboard, registration failures.

**Root Cause:** Incorrect agent configuration pointing to wrong manager IP/port or certificate validation failure.

**Resolution:**

- Verify `WAZUH_MANAGER` environment variable in `ossec.conf`
- Check agent SSL certificate is signed by correct CA
- Confirm port 1514/1515 is accessible from agent network segment
- Restart agents with debug logging enabled for detailed error information

## CHAPTER 8

# Maintenance & Operational Procedures

---

Regular maintenance is essential for ensuring continued platform performance and security. This section outlines standard operational procedures including backup strategies, update procedures, and monitoring configurations.

## Backup Procedures

| Component      | Backup Method                  | Frequency    | RPO Target |
|----------------|--------------------------------|--------------|------------|
| PostgreSQL     | PgBaseBackup + WAL archiving   | Every 15 min | 15 minutes |
| Elasticsearch  | Snapshot to S3/NFS mount       | Hourly       | 1 hour     |
| Kafka Topics   | MirrorMaker to DR cluster      | Real-time    | Near-zero  |
| TheHive/Cortex | Cassandra backup + ES snapshot | Daily        | 24 hours   |
| MISP           | MySQL dump + filesystem backup | Daily        | 24 hours   |
| GRR Artifacts  | Filesystem snapshot to NFS     | Daily        | 24 hours   |

## Update & Upgrade Procedure

- 1 Pre-Upgrade Backup**

Create full backups of all databases and configurations before initiating any upgrade procedure. Verify backup integrity by performing a test restore to a staging environment.
- 2 Rolling Update Strategy**

For stateless services (SOC Platform, API Gateway), use Docker Compose's rolling update feature to deploy new versions with zero downtime. For stateful services (PostgreSQL, Kafka), perform sequential node updates with replication verification between each node.
- 3 Post-Upgrade Verification**

After completing updates, execute the full verification suite from Chapter 6 to confirm all integrations are functioning correctly. Monitor system metrics for at least 24 hours after major version upgrades.

# Ready for Deployment

Your Djezzy National SOC Platform is now ready for production deployment. Follow this guide systematically to ensure a successful implementation of your enterprise-grade security operations center.

---

Document Version: 11.1.0 | Phase 11 Production Build  
Generated: January 2026 | Classification: Internal Use Only